

Roteamento offline em estradas não mapeadas

Do traço extraído por satélite a um grafo roteável no aparelho, sem rede — com confiança declarada por trecho.

Offline routing over unofficial Amazon roads

GeoVicinal • Priscila Motta e Samuel Santos

Geoprocessamento clássico • zero chamadas a LLM ou modelo de visão

github.com/samuel-c-santos/amazoniahack

Um traço extraído não é um grafo

Linhas que se cruzam na tela não compartilham vértice. A detecção do PrevisIA já estava entregue; o que faltava era topologia — e ela precisa ser medida, não estimada no olho.

35.791

traços viários extraídos do Sentinel-2

1.874

componentes desconexos na malha
crua

2,6%

dos nós no maior componente —
número do README reproduzido como
checagem

Topologia

três camadas viárias num
mesmo espaço de nós

Confiança

por aresta, com origem do dado
e custo

Roteamento

Dijkstra avaliado nos 16 pares
do desafio

Entrega offline

grafo binário + roteador
embarcado, sem rede

Preparação online, consulta sem rede

BUILD · 1× POR MUNICÍPIO

OSM 7.095 · IBGE 24.346 · PrevisIA
35.791 linhas

Grafo unificado — mesmo espaço de nós

Snap só em pontas de grau 1 (300 m; 100
m no IBGE)

Validação hidrográfica — drenagem
SIGERH, 57.559 trechos

Tier + custo por aresta — CAR 2.799
polígonos, 287 pontes OSM

serializa



ARTEFATO

paragominas.graph

CSR binário, 29,3 MB

763.856 nós

1.559.340 arestas

consulta



QUERY · OFFLINE, NO APARELHO

Carrega o binário — 101 ms

Índice espacial em UTM — 222
ms

Snap de origem e destino

Dijkstra — dezenas de ms

Rota colorida por tier

Confiança é atributo da aresta

TIER	MULTIPLICADOR DE CUSTO
osm	1.0
ibge	1.2
previsia	1.2
confirmed_bridge	1.5
unconfirmed_bridge	4.0
risky_bridge	removida do grafo

Cruzar rio sem passagem conhecida a menos de 40 m é exclusão categórica, não penalização: uma versão anterior tolerava isso abaixo de 0,2% da rota, e o filtro voltava a falhar em silêncio.

Criadas pelo snap	8.876
Confirmadas pelo CAR	1.098
Excluídas — rio sem ponte	1.168
Rejeitadas — IBGE >100 m	2.845
Componentes antes/depois	2.263 → 704

ERRO DOCUMENTADO

A primeira versão fazia snap em qualquer nó próximo: mais de 4 milhões de arestas espúrias e travamento por memória. A regra de grau 1 é a correção — malha conectada e falsa é o pior resultado possível.

Deslocamento IBGE × OSM: mediana 34 m, p95 424 m, p99 2,5 km — por isso a malha oficial recebe tolerância reduzida e confiança igual à do PrevisIA. Ser oficial não a torna posicionalmente precisa.

Alcance nos 16 pares do desafio

11/16

na malha completa — o README do desafio relatava teto de 9–10/16 só com OSM + PrevisIA

11/16

após remover toda travessia de rio sem passagem conhecida — sem perda de alcance

1/16

se exigirmos confirmação independente pelo CAR em toda ligação

60–90% OSM / IBGE

PrevisIA

Composição típica da rota por distância — o salto de baixa confiança é sempre residual

≤0,5% em ponte não confirmada

Nenhuma das 11 rotas chega a confiança alta: todas dependem de ao menos uma ponte não confirmada (0,2–0,5% da distância) e saem rotuladas como média. É descoberta, não artefato de escala — nenhum par alcançável se resolve inteiramente sobre vias confirmadas.

Pernas a pé declaradas por rota, não desenhadas: destino 298–1.937 m, origem 111–1.444 m, em `final_leg_on_foot_m` e `origin_snap_m`. Sem rota: par-03, 06, 07, 12, 15.

O que roda em campo não precisa de rede

O grafo classificado é serializado em CSR binário e lido por `offline/router.js` — sem dependências, sem chamada externa em tempo de execução. O custo por aresta é o mesmo do pipeline Python: distância × multiplicador de confiança.

A paridade é exata: os 16 pares roteados no navegador reproduzem ponto a ponto os 11/16 do Python. No par-01: 49,1 km, perna de destino 1.937 m, osm 61,9% / ibge 38,0% / ponte não confirmada 0,2%.

A data dos dados por camada vai no cabeçalho do binário — PrevisIA como `null`, declarando a ausência em vez de estimar. Schema completo em `desafio3_grafo_schema.md`.

29,3 MB

grafo inteiro, caber num
Android intermediário

101 ms

para carregar o binário

222 ms

para o índice espacial em UTM

dezenas de ms

por rota, Dijkstra no aparelho

<offline/index.html>

Demo em canvas: malha colorida por tier, rota por dois cliques,
100% offline. O roteador também roda como CLI em Node.

Por que não GeoJSON: só as 11 rotas já ocupam 1,9 MB; o grafo inteiro passaria das centenas de MB. O CSR binário mantém 763.856 nós e 1.559.340 arestas dirigidas em 29,3 MB, carregáveis sem rede.

Custo marginal por rota: zero

Geoprocessamento clássico — NetworkX, Shapely, GeoPandas, SciPy, PyProj — e um roteador em JS. Nenhuma chamada a modelo de linguagem ou de visão, em nenhuma etapa. Sem GPU: o grafo de ≈ 764 mil nós constrói e roteia os 16 pares em poucos minutos num notebook comum.

```
pip install -r requirements.txt
python desafio3_grafo_base.py
python -m http.server 8000
# abrir http://localhost:8000/offline/
```

Versões pinadas em requirements.txt (Python 3.11); o pyproj do pip embute o proj.db. O método replica em outro município trocando o recorte e as consultas WFS.

desafio3_grafo_base.py	Pipeline: grafo, snap, validação, serialização
desafio3_grafo_schema.md	Especificação do formato binário do grafo
offline/router.js	Roteador offline, sem dependências (lib + CLI)
offline/index.html	Demo em navegador: malha por tier + rota por clique
desafio3_avaliacao.md	Avaliação técnica e de confiabilidade
desafio3_changelog.md	Cada correção feita e por quê
fora do repositório	Pacote do organizador, camadas WFS/Overpass e o grafo gerado — não redistribuídos; o script regenera

O que ficou de fora — e o que vem depois

Elevação e declividade	A drenagem serve só ao teste binário de cruzamento; sem DEM nem passabilidade sazonal	Auditar os 5 pares sem rota É o que diz se 11/16 é teto ou piso
Os 5 pares sem rota	Não investigados caso a caso — isolamento real ou limite do snap, não sabemos	Validar ligações com quem conhece o campo 8.876 ligações, 1.098 confirmadas — o resto é pergunta de campo
Acabamento de produto	O demo offline funciona; falta base map georreferenciado (MapLibre/MBTiles) e app Android	Empacotar como app Android O binário já cabe; falta o invólucro e o base map
Marco bônus	Arquitetado, não implementado: sem map-matching de traços de GPS	GPS como terceira fonte Um traço validado vira aresta <code>gps_confirmed</code> no binário, sem reescrever o roteador
Idade da malha PrevisIA	O atributo não existe nos dados; declarado <code>null</code> , não estimado	

Dos 11 pares roteados, 1 se sustenta sobre ligações com confirmação independente. Essa lacuna entre plausível e confirmado é o número mais honesto que temos — e é ele que um agente precisa ver antes de sair.